

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC MỞ THÀNH PHỐ HỒ CHÍ MINH



BÁO CÁO GIỮA KỲ
MÔN HỌC: HỌC SÂU VÀ ỨNG DỤNG
Lớp: MCOM025

HỌC VIÊN THỰC HIỆN: NGUYỄN CHÁNH KHÔI
MSHV: 2584801012002
GIẢNG VIÊN: TS. LÊ VIỆT TUẤN

TP.HCM - 08/2026

Mục lục

1	Giới thiệu bài toán	3
1.1	Bài toán đề ra	3
1.2	Mục tiêu	3
2	Kết quả đạt được	4
2.1	Kết quả Test đạt được khi tự huấn luyện	4
2.2	Kết quả Test ở trên file Instructions của tác giả	4
3	Phân tích vấn đề	5
3.1	Vấn đề trong file Instructions ban đầu	5
3.1.1	Kết quả Test ban đầu trên file	5
3.1.2	Kết quả chạy lại Test trên file	5
3.1.3	Các vấn đề trong file	6
4	Các phương pháp đã dùng	6
4.1	Tóm tắt các phương pháp đã dùng	6
4.2	Chi tiết các phương pháp đã dùng	8
4.2.1	Phần cứng ảo (GPU Tesla T4 của Google Colab)	8
4.2.2	Dữ liệu	8
4.2.3	Huấn luyện mô hình	9

4.2.4	Suy luận	11
-------	--------------------	----

5	Kết luận	11
---	----------	----

1. Giới thiệu bài toán

1.1 Bài toán đề ra

Trong lĩnh vực thị giác máy tính hiện đại, nhận dạng biển báo giao thông quyết định tính an toàn của các phương tiện tự lái và hành khách. Để kiểm tra năng lực hệ thống, bài toán sử dụng bộ dữ liệu GTSRB (German Traffic Sign Recognition Benchmark) với 43 lớp biển báo xuất hiện trong nhiều trường hợp khác nhau.

Bài toán áp dụng mạng Vision Transformer (ViT) với cơ chế Self-attention cho phép quan sát toàn cục bức ảnh nhưng lại rất kém trong việc phân biệt các biển báo có hình dáng tương đồng hay bị nhiễu bởi môi trường.

Các kiến trúc LNL (Locality-iN-Locality) và LNL-MoEx (Moment Exchange) tồn tại để cho ViT khả năng quan sát toàn cục và cục bộ. Đặc biệt biến thể LNL-MoEx còn kỹ thuật trao đổi phân phối dữ liệu để các mạng nơ-ron phải nhận diện biển báo dựa trên khối vật lý, tránh tình trạng overfit về màu sắc hay độ chói của nền ảnh.

1.2 Mục tiêu

Bài báo cáo đề ra mục tiêu ứng dụng những thành tựu đã đạt được của các tác giả bài báo, vận dụng các kiến thức Deep Learning cùng sự hỗ trợ của AI, thực hiện phần huấn luyện mô hình nhằm đẩy hiệu năng của mô hình để đạt mốc chính xác cao nhất (Top-1 Accuracy) 99.6%.

2. Kết quả đạt được

2.1 Kết quả Test đạt được khi tự huấn luyện

```
Test Accuracy (khôi_model_epoch_36.pth): 99.57%
Test Accuracy (khôi_model_epoch_37.pth): 99.70%
Test Accuracy (khôi_model_epoch_38.pth): 99.74%
Test Accuracy (khôi_model_epoch_39.pth): 99.71%
Test Accuracy (khôi_model_epoch_40.pth): 99.75%
-----
TRỌNG SỐ TỐT NHẤT: 99.75% tại file: khôi_model_epoch_40.pth
```

Figure 1: Kết quả kiểm thử mô hình tự huấn luyện, đạt độ chính xác cao nhất 99.75%.

2.2 Kết quả Test ở trên file Instructions của tác giả

```
Vui lòng ấn vào nút 'Choose Files' bên dưới để chọn file .pth từ máy tính:
Choose Files khôi_model...poch_40.pth
khôi_model_epoch_40.pth(n/a) - 24671421 bytes, last modified: 8/5/2026 - 100% done
Saving khôi_model_epoch_40.pth to khôi_model_epoch_40 (1).pth

Đang nạp trọng số từ file: khôi_model_epoch_40 (1).pth...
Đang load checkpoint...
Nạp trọng số thành công!
Load model thành công! Đang tiến hành test trên tập dữ liệu...

Standard accuracy: 99.71 %
```

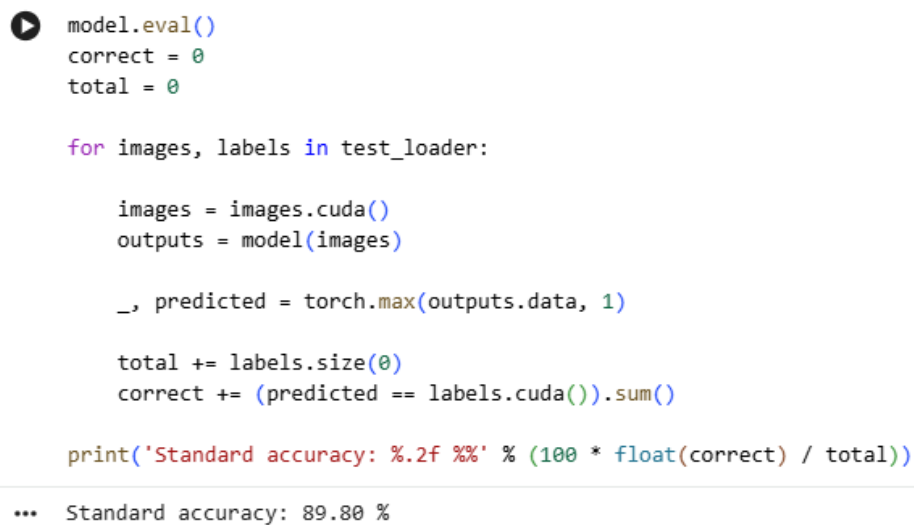
Figure 2: Kết quả kiểm thử trên file Instructions của tác giả, đạt độ chính xác 99.71%.

3. Phân tích vấn đề

3.1 Vấn đề trong file Instructions ban đầu

3.1.1 Kết quả Test ban đầu trên file

Test



```

model.eval()
correct = 0
total = 0

for images, labels in test_loader:

    images = images.cuda()
    outputs = model(images)

    _, predicted = torch.max(outputs.data, 1)

    total += labels.size(0)
    correct += (predicted == labels.cuda()).sum()

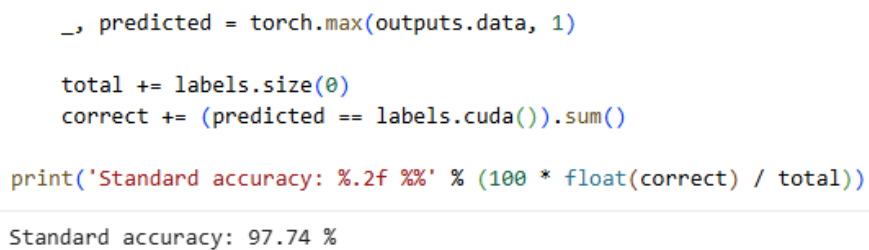
print('Standard accuracy: %.2f %%' % (100 * float(correct) / total))

... Standard accuracy: 89.80 %

```

Figure 3: Kết quả kiểm thử có sẵn trên file Instructions của tác giả.

3.1.2 Kết quả chạy lại Test trên file



```

_, predicted = torch.max(outputs.data, 1)

total += labels.size(0)
correct += (predicted == labels.cuda()).sum()

print('Standard accuracy: %.2f %%' % (100 * float(correct) / total))

Standard accuracy: 97.74 %

```

Figure 4: Kết quả kiểm thử chạy lại trên file Instructions của tác giả.

3.1.3 Các vấn đề trong file

- **Chưa thực hiện Cường hóa dữ liệu**

Các phép biến đổi hình học có trong file đã bị tác giả comment. Dữ liệu đưa vào mạng chỉ trải qua thao tác thay đổi kích thước và chuyển đổi tensor. Việc học trên hình ảnh tĩnh khiến mạng nơ-ron nhanh bị overfit dữ liệu huấn luyện thay vì học được bản chất của các biển báo.

- **Kích thước Batch Size quá nhỏ**

Với một kiến trúc Transformer, `batch_size = 15` là quá nhỏ. Thiết lập này tạo ra gradient có độ nhiễu cao sau mỗi lần cập nhật trọng số, khiến hàm mất mát hội tụ chậm và chưa tận dụng tốt năng lực tính toán của phần cứng.

- **Bỏ qua sự mất cân bằng dữ liệu**

Lệnh gọi `DataLoader` cho `train_loader` chỉ dùng `shuffle=True`, trong khi tập GTSRB vốn bị mất cân bằng lớp, khiến mô hình ưu tiên các lớp có nhiều dữ liệu và dễ dự đoán sai các lớp ít dữ liệu hơn.

- **Xung đột logic trong Learning Rate Scheduler**

Mã nguồn gốc thiết lập vòng lặp huấn luyện với `num_epochs = 5` và bộ điều chỉnh tốc độ học `StepLR` với chu kỳ `step_size = 30`, tức là sau 30 epoch mới giảm tốc độ học. Vì quá trình huấn luyện chỉ kéo dài 5 epoch, tốc độ học không được điều chỉnh giảm để tinh chỉnh trọng số ở những epoch cuối.

4. Các phương pháp đã dùng

4.1 Tóm tắt các phương pháp đã dùng

Table I: Bảng tóm tắt các phương pháp tối ưu hóa toàn diện

Khía cạnh Tối ưu	Hiện trạng mã nguồn gốc	Phương pháp áp dụng	Lý do & Hiệu quả đạt được
Phần cứng	Dùng FP32 ngôn dụng lượng và bằng thông, vô hiệu hóa Tensor Cores, Batch Size giới hạn ở 15.	Mixed Precision Training (MPT) kết hợp tăng Batch Size lên 64 .	Kích hoạt Tensor Cores tính toán nhanh gấp 2-3 lần. autocast ép phép toán nặng về FP16 (giảm 50% VRAM). GradScaler ngăn lỗi Underflow. VRAM thừa cho phép tăng Batch Size.
Dữ liệu	Vô hiệu hóa biến đổi hoặc thiết lập các phép tạo nhiễu với xác suất thủ công.	Tích hợp TrivialAugment Wide .	Tự động thay đổi cường độ méo mó liên tục, ép mô hình học bản chất vật lý thực tế của biển báo, triệt tiêu hiện tượng học vẹt (Overfitting).
Dữ liệu	Lấy mẫu ngẫu nhiên cơ bản (shuffle=True), bỏ qua mất cân bằng dữ liệu.	Sử dụng WeightedRandom Sampler .	Buộc bộ nạp ưu tiên các ảnh thuộc nhóm thiểu số với tần suất cao hơn, giải quyết sự thiên vị phân lớp của tập GTSRB.
Dữ liệu	Chỉ sử dụng 1 luồng duy nhất, phát sinh độ trễ (GPU phải chờ CPU).	Thiết lập num_workers=2 .	Tiền xử lý dữ liệu và đọc ảnh đa luồng dưới nền. GPU hoạt động liên mạch ở công suất tối đa, loại bỏ thời gian chết.
Dữ liệu	Đưa trực tiếp dữ liệu ảnh thô (pixel mang giá trị dương) vào mạng.	Kích hoạt Normalize với hệ số 0.5.	Đưa phân phối điểm ảnh về không gian toán học đối xứng $[-1, 1]$, giúp đạo hàm mượt mà và ổn định cập nhật trọng số.
Huấn luyện	Hàm CrossEntropyLoss thô và không ngắt kết nối nơ-ron.	Label Smoothing và drop_path_rate .	Chống tự tin thái quá (Loss không bao giờ chạm 0) và chống hiện tượng đồng thích nghi nơ-ron. Ép mô hình phân tích đặc trưng độc lập và khái quát.
Huấn luyện	Thuật toán SGD tốc độ hội tụ chậm và khó điều hướng.	Tối ưu hóa bằng AdamW .	Tách biệt hoàn toàn thành phần phạt suy giảm trọng số (decoupled weight decay), giúp chuẩn hóa chuẩn xác và hội tụ ổn định.

Tiếp tục ở trang sau

Table I – tiếp theo

Khía cạnh Tối ưu	Hiện trạng mã nguồn gốc	Phương pháp áp dụng	Lý do & Hiệu quả đạt được
Huấn luyện	Tốc độ học (LR) cố định hoặc giảm đột ngột theo bậc thang.	Điều hướng chiến lược bằng OneCycleLR .	Quỹ đạo Parabol: Bứt tốc để văng khỏi các điểm cực tiểu cục bộ hẹp, hãm phanh tiệm cận 0 để siêu tinh chỉnh. Kích hoạt hiện tượng "Siêu hội tụ".
Suy luận	Đánh giá một lần duy nhất trên ảnh gốc (dễ sai nếu ảnh nhiễu/góc khó).	Kỹ thuật Test-Time Augmentation (TTA) .	Đánh giá đa góc nhìn ảo và lấy trung bình cộng xác suất (Ensemble). Tự động khử nhiễu cục bộ, vớt ảnh khó để đẩy điểm lên 99.75%.

4.2 Chi tiết các phương pháp đã dùng

4.2.1 Phần cứng ảo (GPU Tesla T4 của Google Colab)

- **Mixed Precision Training (MPT)**

Để khai thác nhiều nhất có thể các lõi Tensor Cores trên GPU Tesla T4 và tiết kiệm bộ nhớ, kỹ thuật MPT được ứng dụng để tối ưu tính toán và tăng kích thước batch.

Lệnh autocast ép các phép toán ma trận nặng xuống định dạng FP16 giúp tính nhanh, giảm 50% VRAM sử dụng nhưng vẫn giữ các phép toán nhạy cảm ở FP32 để bảo toàn độ chính xác. Cùng lúc, GradScaler tăng hàm Loss lên cao trong pha đạo hàm ngược để ngăn các Gradient siêu nhỏ bị lỗi Underflow của FP16 làm tròn về 0.

Lượng VRAM được giải phóng nhờ FP16 cho phép đẩy mạnh kích thước batch từ 15 lên 64 ảnh, tăng tốc độ hội tụ.

4.2.2 Dữ liệu

- **TrivialAugmentWide** thay cho các tăng cường dữ liệu có sẵn

Với ý tưởng sử dụng 1 dạng tăng cường dữ liệu cho mỗi vòng lặp trong danh sách các phép biến hóa của tác giả, TrivialAugmentWide được dùng như một phương án thay thế.

Thay vì tự thiết kế từng phép biến đổi hình học/màu sắc và gán xác suất cố định bằng tay như mã nguồn gốc, `TrivialAugmentWide` tự động thay đổi mức độ biến dạng liên tục thay vì chỉ dùng một mức nhiễu tĩnh.

Việc liên tục nạp vào các hình ảnh với cường độ méo mó đa dạng sẽ buộc mạng Transformer không thể overfit trên các dữ liệu sạch, mà phải tập trung trích xuất cấu trúc vật lý thực tế của dữ liệu biến báo.

- **WeightedRandomSampler**

Tập dữ liệu GTSRB bị mất cân bằng số lượng mẫu nghiêm trọng. Mã nguồn gốc bỏ qua vấn đề này, chỉ dùng cơ chế xáo trộn ngẫu nhiên, dễ bị thiên vị các lớp đa số so với lớp thiểu số.

`WeightedRandomSampler` buộc bộ nạp dữ liệu `DataLoader` ưu tiên lấy các bức ảnh thuộc nhóm thiểu số với tần suất cao hơn để bù đắp cho sự phân bố không đều về lớp dữ liệu, giúp cải thiện độ chính xác đồng đều trên toàn bộ 43 lớp biến báo.

- **num_workers = 2**

Ở mã nguồn gốc, `DataLoader` chỉ dùng 1 luồng duy nhất để đọc và tiền xử lý ảnh, dẫn đến phát sinh độ trễ, GPU phải đợi CPU cấp dữ liệu.

Đặt `num_workers = 2` cho phép CPU kích hoạt các tiến trình con cho phép thao tác đọc ảnh từ ổ cứng và áp dụng các tăng cường được thực hiện song song dưới nền. Với dữ liệu luôn được chuẩn bị sẵn sàng và đưa liên tục vào bộ nhớ VRAM, GPU sẽ phải hoạt động ở công suất tối đa, loại bỏ thời gian chết và rút ngắn tổng thời gian huấn luyện.

4.2.3 Huấn luyện mô hình

- **Thêm Label Smoothing**

Hàm `CrossEntropyLoss` mặc định thường ép mô hình dự đoán xác suất tuyệt đối (1.0 cho nhãn đúng và 0.0 cho nhãn sai), dẫn đến mạng nơ-ron dễ bị kiêu ngạo, dễ học vẹt và nhạy cảm với nhiễu.

Label Smoothing giữ cho giá trị Loss không bao giờ chạm mức 0.0, buộc mô hình phải học các đặc trưng khái quát thay vì cố gắng học thuộc lòng dữ liệu.

- **drop_path_rate**

Trong các mạng có độ sâu lớn như kiến trúc Transformer, các nơ-ron có xu hướng ỷ lại và phụ thuộc lẫn nhau, làm mô hình mất đi tính linh hoạt trước dữ liệu thực tế mới.

Thông số `drop_path_rate` can thiệp trực tiếp vào cấu trúc của `LNL_Ti` bằng cách ngắt bỏ ngẫu nhiên toàn bộ một số đường truyền nhánh trong quá trình lan truyền tiến. Việc một số khối bị ngắt kết nối đột ngột sẽ khiến các mạng còn lại phải tự học phân tích đặc trưng một cách độc lập, giúp tổng thể kiến trúc chịu lỗi tốt hơn.

- **AdamW**

Mã nguồn gốc sử dụng SGD cơ bản với tốc độ hội tụ chậm và thường gặp khó trong điều hướng không gian đặc trưng phức tạp của các kiến trúc mạng Transformer.

`AdamW` tách biệt hoàn toàn thành phần phạt suy giảm trọng số ra khỏi bước tính toán đạo hàm Gradient, giúp việc chuẩn hóa diễn ra chuẩn xác hơn, ép các trọng số không phình to quá mức mà không làm sai lệch hướng đi của Gradient.

Nhờ khả năng tự động điều chỉnh kích thước bước nhảy (learning rate) linh hoạt cho từng siêu tham số, `AdamW` giúp kiến trúc `LNL_Ti` hội tụ ổn định hơn. Đặc biệt, khi được kết hợp với bộ lập lịch `OneCycleLR`, sức mạnh của nó được phát huy tối đa.

- **OneCycleLR**

Phương pháp cũ thường giữ tốc độ học ở một mức cố định từ đầu đến cuối, hoặc giảm đột ngột theo từng bậc thang. Vấn đề của phương pháp này là nếu tốc độ quá nhỏ, tốc độ học rất chậm và dễ bị mắc kẹt lại ở các cực tiểu cục bộ; nhưng nếu tốc độ quá lớn, việc học sẽ liên tục văng khỏi đích đến mà không thể dừng lại đúng điểm sâu nhất.

`OneCycleLR` thiết kế một quỹ đạo tốc độ học linh hoạt trong cùng một chu kỳ. Nửa đầu có tốc độ học chậm, đi dần lên mức tối đa, giúp mô hình thoát khỏi các điểm cực tiểu cục bộ chật hẹp, buộc mạng lưới khám phá không gian rộng hơn để tìm ra vùng tối ưu và an toàn.

Khi đã đến đúng vùng tối ưu, tốc độ học sẽ giảm dần về tiệm cận 0. Lúc này, mô hình sẽ bước những bước cực nhỏ để tinh chỉnh và nằm gọn vào điểm hội tụ hoàn hảo mà không bị trượt ra ngoài.

Việc thay đổi tốc độ học linh hoạt theo đồ thị chống học vệt tốt, giúp mô hình chốt hạ điểm số một cách chính xác, tiết kiệm tối đa thời gian thay vì phải dò dẫm như phương pháp cũ.

4.2.4 Suy luận

- **Test-Time Augmentation (TTA)**

Ở mã nguồn gốc, tại khâu kiểm thử, một bức ảnh chỉ được đưa vào mạng nơ-ron một lần duy nhất để đưa ra quyết định, nếu ảnh bị mờ, nhiễu hay chụp ở một góc khó, mô hình sẽ đưa ra nhận diện sai mà không có cơ hội để kiểm tra lại để sửa.

TTA can thiệp vào khâu kiểm thử, biến đổi hình ảnh trước khi đánh giá. Từ một bức ảnh ban đầu, TTA tạo ra thêm một vài phiên bản ảo bằng một vài phép biến đổi đơn giản (phóng to, thu nhỏ, xoay, dịch). Lưới nơ-ron sẽ đưa ra dự đoán trên tất cả các phiên bản này rồi dùng thuật toán Ensemble để lấy trung bình cộng xác suất làm đáp án cuối cùng. Việc lấy kết quả từ nhiều góc nhìn giúp mô hình tự khử nhiễu cục bộ, có thể dự đoán tốt các bức ảnh rác hoặc quá khó, giúp làm tăng độ chính xác của mô hình.

5. Kết luận

Quá trình phân tích và tối ưu hóa hệ thống nhận dạng biển báo giao thông đã đạt độ chính xác từ 89.80% lên 99.75%. Thành quả đột phá này là kết tinh của việc tái cấu trúc huấn luyện thông qua hàng loạt các kỹ thuật tiên tiến: triệt tiêu hiện tượng học vẹt và khắc phục mất cân bằng phân lớp bằng `TrivialAugmentWide` và `WeightedRandomSampler`; ổn định quá trình cập nhật trọng số nhờ chuẩn hóa đối xứng, thuật toán `AdamW` kết hợp quỹ đạo siêu hội tụ `OneCycleLR`; gia tăng tính bền vững của mạng bằng Label Smoothing và ngắt nơ-ron ngẫu nhiên (`drop_path_rate`). Đồng thời, hệ thống đã khai thác triệt để sức mạnh phần cứng bằng Mixed Precision Training, kết hợp cùng kỹ thuật đánh giá đa góc nhìn Test-Time Augmentation (TTA) ở khâu suy luận để khử nhiễu cục bộ và trích xuất điểm đặc trưng chính xác nhất. Tất cả những phương thức này đã biến một mã nguồn chưa tối ưu thành hệ thống thị giác máy tính đáp ứng yêu cầu khắc khe của bài toán.

GHI CHÚ

Github: https://github.com/KhoiNguyenChanh/ChanhKhoi_DLA_MidTerm

CÁC FILE TRONG GITHUB:

- DLAdvanced_GK_Instructions_Submit: File .ipynb train mô hình.
- DLAdvanced_GK_Instructions_S_Test: File .ipynb chạy test.
- khoi_model_epoch_40.pth: file trọng số tốt nhất đạt được kết quả tốt nhất.